

# 컴퓨터 구조론

## HW3 Report



32217072 모바일시스템공학과 김도익  
컴퓨터 구조론 HW3

# 목차

## 1. 서론

## 2. 본론

### 1) 배경지식

- 파이프라인이란

- hazard

  - 1.data hazard

  - 2.structure hazard

  - 3.control hazard

### 2) 디자인

### 3) 코드설명

### 4) 구현

### 5) 프로그램 실행

## 3. 결론

## 1. 서론

- 이 레포트는 MIPS를 구현한 코드를 바탕으로 Pipelined된 Datapath의 작동 방식과 각 명령어의 실행 과정을 설명한다. 앞선 hw2 single-cycle design에서는 clock cycle을 load 명령어에 맞춰야 하기 때문에 성능의 저하가 있었다. 이 레포트에는 파이프라인의 작동, 구현을 함으로서 성능을 어떻게 향상시켰는지에 대해 설명한다.

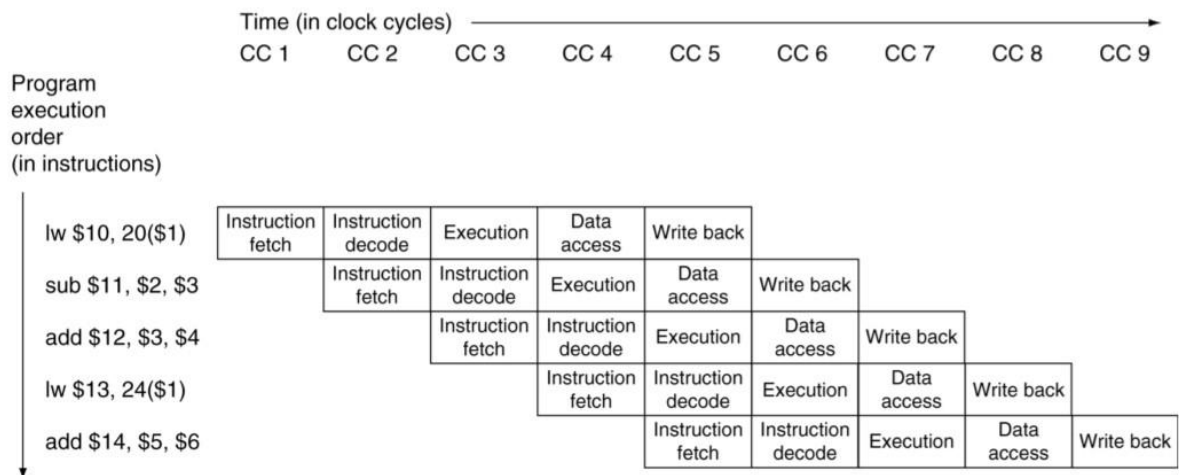
## 2. 본론

### 1) 배경지식

#### ● Pipeline이란

- 파이프라인이란 프로세서에서 성능을 높이기 위해서 명령어 처리 과정으로 명령어 처리를 여러 단계로 나누어 단계별로 동시에 수행하여 병렬화를 시키는 것을 말한다. 파이프라인은 다음과 같은 다섯 단계로 구성된다.

1. IF(Instruction Fetch): 메모리에서 명령어를 가져온다.
2. ID(Instruction Decode): 명령어를 읽고/해독, 레지스터를 읽는다.
3. EX(Execute): 연산을 수행하거나 주소를 계산한다.
4. MEM(Memory access): 데이터 메모리에 접근하여 데이터를 읽거나 쓴다.
5. WB(Write back): 최종 결과를 레지스터에 쓴다.



Pipeline은 처리 속도를 향상시킬 수 있지만 여러 hazard들을 고려하지 않는다면 성능을 떨어뜨릴 수 있기 때문에 hazard들을 올바르게 처리하는 것이 중요하다.

## ● Hazard

### 1. Structural Hazard

Structural hazard는 Structural Hazard이다. Structural Hazard는 어떤 resource를 써야하는데, 누가 지금 사용하고 있을 때 발생한다. 이러한 문제는 간단하게 single memory 대신 instruction memory와 data memory로 분할하면 된다.

### 2. Data Hazard

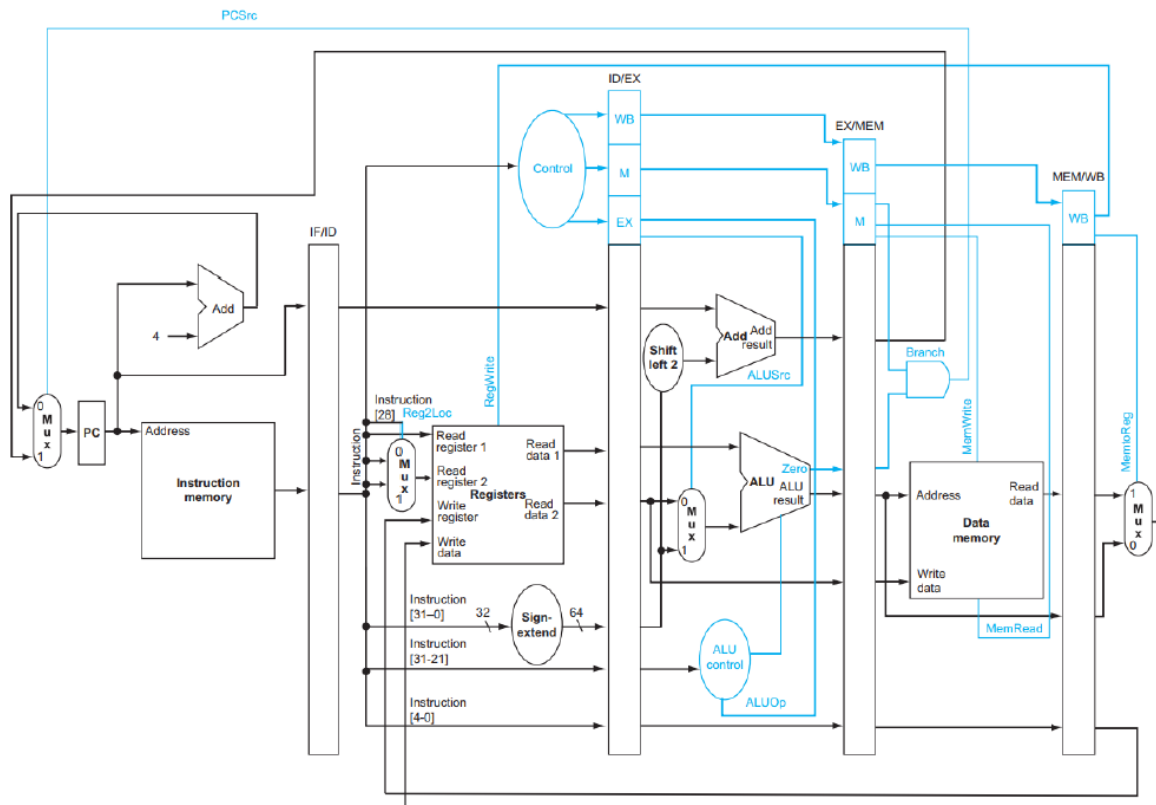
Data Hazard는 어떤 stage가 다른 stage가 끝날 때까지 기다려야 하기 때문에 발생한다. 즉 instruction이 이전의 instruction에 대해 dependency를 가지고 있기 때문에 발생한다. 이러한 문제는 컴파일러가 대부분 해결해준다. 하지만 이러한 dependency는 자주 발생하고, 컴파일러가 해결하지 못한다면 손해가 크다. 따라서 하드웨어적으로 문제를 해결하는데 이것이 forwarding이다. Ex가 끝나자마자 wiring을 통해 바로 다음 명령어에게 데이터를 전달해주는 것이다.

### 3. Control Hazard

Control Hazard는 conditional branch를 통해 pc값을 바꿀 때, 이미 pipeline에 들어와 있는 명령어가 flush 되는 현상이다. 즉 pipeline에 이미 반입된 다음 명령어를 버려야 한다.

다는 뜻이다. 이런 무의미한 명령어를 줄이는 것이 문제이다. 이 문제를 해결하기 위해 분기 결과를 예측하는 방법인 Branch prediction이 있다.

## 2) 디자인



이 코드는 MIPS 명령어 집합을 시뮬레이션하는 파이프라인 프로세서의 간단한 구현이다. 파이프라인 프로세서 디자인은 명령어를 동시에 여러 단계로 나누어 실행하여 CPU 성능을 향상시키는 것을 목표로 한다. 이 코드는 5단계 파이프라인을 사용하며, 각 단계는 IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access), WB (Write Back)로 구성된다.

### 3) 코드설명

```
// pipelined registers
typedef struct {
    unsigned int pc;
    unsigned int instruction;
} IF_ID;

typedef struct {
    unsigned int pc;
    unsigned int instruction;
    int opcode;
    int rs;
    int rt;
    int rd;
    int shamt;
    int funct;
    int constant;
    int address;
    char type;
    control CU;
} ID_EX;

typedef struct {
    unsigned int pc;
    int ALUresult;
    int rs;
    int rt;
    int rd;
    int constant;
    int address;
    char type;
    control CU;
} EX_MEM;

typedef struct {
    unsigned int pc;
    int ALUresult;
    int memoryValue;
    int rd;
    int rt;
    char type;
    control CU;
} MEM_WB;
```

**파이프라인 레지스터:** 각 파이프라인 단계 사이에 파이프라인 레지스터(IF/ID, ID/EX, EX/MEM, MEM/WB)를 사용하여 데이터와 제어 신호를 저장하고 전달한다. 이러한 레지스터는 파이프라인의 각 단계가 독립적으로 작동할 수 있도록 하며, 파이프라인 단계 사이의 데이터를 보호한다.

```
// 명령어를 가져오는 함수
void IF() {
    ifid[1].instruction = instMemory[pc / 4];
    ifid[1].pc = pc;
    printf("[IF: 0x%08X]\n", ifid[1].instruction);
    printf("[PC Update] 0x%08X -> 0x%08X\n", pc, pc + 4);
    pc = Adder(pc, 1); // 다음 명령어를 실행할 준비
}
```

**IF:** 이 함수는 프로그램 카운터(PC)를 이용해 명령어 메모리에서 명령어를 읽고, 다음 명령어를 위해 PC를 증가시킨다. 읽어들인 명령어와 PC 값은 IF\_ID 레지스터에 저장된다.

```

// 명령어 디코드 함수
void ID() {
    idex[1].pc = ifid[1].pc;
    idex[1].instruction = ifid[1].instruction;

    // opcode: 상위 6비트 추출
    idex[1].opcode = (idex[1].instruction >> 26) & 0x3F;

    if ((idex[1].instruction >> 26) == 0x00) { // R 타입 명령어
        idex[1].rs = (idex[1].instruction >> 21) & 0x1F;
        idex[1].rt = (idex[1].instruction >> 16) & 0x1F;
        idex[1].rd = (idex[1].instruction >> 11) & 0x1F;
        idex[1].shamt = (idex[1].instruction >> 6) & 0x1F;
        idex[1].funct = idex[1].instruction & 0x3F;
        idex[1].type = 'R'; // 전역 변수 type 설정
        R_count++;
    }
    else if ((idex[1].instruction >> 26) == 0x02 || (idex[1].instruction >> 26) == 0x03) { // J 타입 명령어
        idex[1].address = idex[1].instruction & 0x3FFFFFFF;
        idex[1].type = 'J'; // 전역 변수 type 설정
        J_count++;
    }
    else { // I 타입 명령어
        idex[1].rs = (idex[1].instruction >> 21) & 0x1F;
        idex[1].rt = (idex[1].instruction >> 16) & 0x1F;
        idex[1].constant = SignExtend(idex[1].instruction & 0xFFFF);
        idex[1].type = 'I'; // 전역 변수 type 설정
        I_count++;
    }

    CU_signal(&idex[1].CU, idex[1].opcode, idex[1].type); // control unit 값 설정
}

```

```

// Branch decision
if (idex[1].type == 'I' && idex[1].opcode == 0x04) { // BEQ
    if (Reg[idex[1].rs] == Reg[idex[1].rt]) {
        pc = branchAdder(idex[1].pc, idex[1].constant);
        // Flush the pipeline
        memset(&ifid[1], 0, sizeof(IF_ID));
        memset(&idex[1], 0, sizeof(ID_EX));
        return;
    }
}
else if (idex[1].type == 'I' && idex[1].opcode == 0x05) { // BNE
    if (Reg[idex[1].rs] != Reg[idex[1].rt]) {
        pc = branchAdder(idex[1].pc, idex[1].constant);
        // Flush the pipeline
        memset(&ifid[1], 0, sizeof(IF_ID));
        memset(&idex[1], 0, sizeof(ID_EX));
        return;
    }
}
}

```

ID: 이 함수는 명령어를 해독하여 opcode와 관련된 필드를 추출한다. 또 명령어의 타입에 따라 제어 신호를 설정한다. 제어 신호는 파이프라인의 각 단계에서 사용된다.

```

// 명령어 분류 및 실행
void EX() {
    exmem[1].pc = idex[1].pc;
    exmem[1].CU = idex[1].CU;
    exmem[1].rs = idex[1].rs;
    exmem[1].rt = idex[1].rt;
    exmem[1].rd = idex[1].rd;
    exmem[1].constant = idex[1].constant;
    exmem[1].address = idex[1].address;
    exmem[1].type = idex[1].type;

    // Forwarding unit for EX hazard detection
    int forwardA = 0;
    int forwardB = 0;

    if (exmem[0].CU.RegWrite && exmem[0].rd != 0 && exmem[0].rd == idex[1].rs) {
        forwardA = 1;
    }
    if (exmem[0].CU.RegWrite && exmem[0].rd != 0 && exmem[0].rd == idex[1].rt) {
        forwardB = 1;
    }

    if (memwb[0].CU.RegWrite && memwb[0].rd != 0 && memwb[0].rd == idex[1].rs) {
        forwardA = 2;
    }
    if (memwb[0].CU.RegWrite && memwb[0].rd != 0 && memwb[0].rd == idex[1].rt) {
        forwardB = 2;
    }

    int operand1 = (forwardA == 2) ? memwb[0].ALUresult : (forwardA == 1) ? exmem[0].ALUresult : Reg[idex[1].rs];
    int operand2 = (forwardB == 2) ? memwb[0].ALUresult : (forwardB == 1) ? exmem[0].ALUresult : Reg[idex[1].rt];

    int validALUresult = 0; // ALUresult 유효성 확인 변수
}

```

EX:

이함수에서 데이터 해저드를 해결하기 위해 forwarding 유닛을 구현했다.

forwardA 와 forwardB 변수를 통해 현재 명령어가 이전 명령어에서 값을 전달받을지 결정한다

EX/MEM 단계와 MEM/WB 단계의 레지스터와 현재 명령어의 레지스터를 비교하여 Forwarding 여부를 결정한다.

operand1와 operand2를 결정한다. 이 값들은 Forwarding 유닛을 통해 전달된 값일 수도 있고, 레지스터 파일에서 읽어온 값일 수도 있다.



```

// memory access
void MEM() {
    memwb[1].pc = exmem[1].pc;
    memwb[1].ALUresult = exmem[1].ALUresult;
    memwb[1].CU = exmem[1].CU;
    memwb[1].rd = exmem[1].rd;
    memwb[1].rt = exmem[1].rt;
    memwb[1].type = exmem[1].type;

    printf("[MEM: 0x%08X]\n", exmem[1].pc);
    unsigned int address;
    unsigned int value;

    if (exmem[1].CU.MemRead == 1) {
        address = exmem[1].ALUresult;
        memwb[1].memoryValue = instMemory[address / 4];
        printf("Load, Address: 0x%08X, Value: %d\n", address, memwb[1].memoryValue);
    }
    else {
        printf("Pass\n");
    }

    if (exmem[1].CU.MemWrite == 1) {
        address = exmem[1].ALUresult;
        value = Reg[exmem[1].rt];
        instMemory[address / 4] = value;
        printf("Store, Address: 0x%08X, Value: %d\n", address, value);
    }
    else {
        printf("Pass\n");
    }

    printf("Branch: %d, MemRead: %d, MemWrite: %d, RegWrite: %d, MemtoReg: %d\n", exmem[1].Branch,

```

MEM:

## 1. 제어 신호 및 기타 데이터 전송

memwb[1].pc, memwb[1].CU, memwb[1].rd, memwb[1].rt, memwb[1].type 에 EX 단계에서 가져온 값을 저장한다.

## 2. 메모리 읽기

exmem[1].CU.MemRead 신호가 1 인 경우, 즉 메모리 읽기 명령어인 경우 exmem[1].ALUresult 주소의 메모리 값을 읽어 memwb[1].memoryValue 에 저장한다.

## 3. 메모리 쓰기

exmem[1].CU.MemWrite 신호가 1 인 경우, 즉 메모리 쓰기 명령어인 경우 exmem[1].ALUresult 주소에 Reg[exmem[1].rt] 값을 저장한다.

```

// writeback
void WB() {
    printf("[WB: 0x%08X]\n", memwb[1].pc);
    switch (memwb[1].type) {
        case 'R':
            if (memwb[1].CU.RegWrite == 1) {
                Reg[memwb[1].rd] = memwb[1].ALUresult;
                printf("Write Register: %s, Value: %d\n", register_names[memwb[1].rd], Reg[memwb[1].rd]);
            }
            break;
        case 'I':
            if (memwb[1].CU.RegWrite == 1) {
                if (memwb[1].CU.MemtoReg == 1) {
                    Reg[memwb[1].rt] = memwb[1].memoryValue;
                }
                else {
                    Reg[memwb[1].rt] = memwb[1].ALUresult;
                }
                printf("Write Register: %s, Value: %d\n", register_names[memwb[1].rt], Reg[memwb[1].rt]);
            }
            break;
        case 'J':
            printf("Write Register: $ra, Value: %d\n", pc);
            break;
        default:
            break;
    }
    printf("RegWrite: %d, MemtoReg: %d\n", memwb[1].CU.RegWrite, memwb[1].CU.MemtoReg);
}

```

WB:

## 제어 신호 및 기타 데이터 전송

memwb[1].pc, memwb[1].CU, memwb[1].rd, memwb[1].rt, memwb[1].type 에 MEM 단계에서 가져온 값을 저장한다.

## 레지스터 파일 갱신

memwb[1].CU.RegWrite 신호가 1인 경우, 즉 레지스터 파일에 기록해야 하는 경우 memwb[1].type 에 따라 적절한 레지스터를 갱신한다.

R 타입 명령어의 경우 memwb[1].rd 레지스터에 memwb[1].ALUresult 값을 저장한다.

I 타입 명령어의 경우 memwb[1].rt 레지스터에 memwb[1].memoryValue 또는 memwb[1].ALUresult 값을 저장한다.

J 타입 명령어의 경우 반환 주소 레지스터 \$ra 를 갱신한다.

#### 4) 구현

| Implemented parts | o/x |
|-------------------|-----|
| fetch             | O   |
| decode            | O   |
| execute           | O   |
| Memory access     | O   |
| Write back        | O   |
| MUX               | X   |

##### - MUX의 구현실패

본 프로그램에서 MUX를 구현하는 데 어려움을 겪었다. MUX는 cpu 설계에서 여러 입력 중 하나를 선택하여 출력으로 보내는 중요한 역할을 한다. 그러나 MUX는 여러 신호를 제어하기 위해 복잡한 로직이 필요하다. 각 입력 신호가 올바른 조건에서 선택되어야 하여, 이 조건들을 정확히 구현하는 데 어려움이 있었다.

##### - Write back구현

Write back 단계를 별도의 함수로 분리하여 구현하고자 하였으나 파이프라인의 각 단계가 밀접하게 연결되어 있어, Write

back단계를 분리하는데 실패하였다.

## 5) 프로그램 실행

### KOI에서 프로그램실행 방법

- ① 터미널을 열고 `gcc -o 32217072 32217072.c` 커맨드를 작성하여 컴파일 한다.
- ② bin파일을 준비한다. 파일이름은 test로 가정한다.
- ③ `./32217072 test.bin` 커맨드를 작성한다.

### KOI에서 프로그램 실행 출력

다음 이미지는 HW2 input1 ~ 5를 순서대로 사용

#### 1.sum.bin

```
[EX: 0x8FC20004]
data1: 16777200, data2: -620773486 (forwarding), ALU result: 16777204
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 1, MemWrite: 0, RegWrite: 1, MentoReg: 1
[ID: 0x00000000]
Type: R, Inst: lw $zero $zero $zero
opcode: 0x0, rs: 0 (value), rt: 0 (value), rd: 0, shamt: 0, funct: 0x0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MentoReg: 0
[IF: 0x24420001]
[PC Update] 0x00000038 -> 0x0000003C
32217072> cycle: 19493
[WB: 0x0000002C]
Write Register: $v0, Value: 16777200
RegWrite: 1, MentoReg: 0
[MEM: 0x00000030]
Load, Address: 0x00FFFFF4, Value: 16777205
Pass
Branch: 0, MemRead: 1, MemWrite: 0, RegWrite: 1, MentoReg: 1
[EX: 0x00000000]
data1: 16777204, data2: 16777204 (forwarding), ALU result: 16777204
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MentoReg: 0
[ID: 0x24420001]
Type: I, Inst: sll $v0, 1($v0)
opcode: 0x9, rs: 2 (value), rt: 2, imm: 0x1
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MentoReg: 0
[IF: 0xAFC20004]
[PC Update] 0x0000003C -> 0x00000040
32217072> cycle: 19494
[WB: 0x00000030]
Write Register: $v0, Value: 16777205
RegWrite: 1, MentoReg: 1
[MEM: 0x00000034]
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MentoReg: 0
[EX: 0x24420001]
data1: 16777204, data2: 16777204 (forwarding), ALU result: 16777205
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MentoReg: 0
[ID: 0xAFC20004]
Type: I, Inst: addiu $v0, 4($s8)
opcode: 0x2B, rs: 30 (value), rt: 2, imm: 0x4
```

## 2.func.bin

```
Store, Address: 0x00FFA8D0, Value: 16754904
Branch: 0, MemRead: 0, MemWrite: 1, RegWrite: 1, MemtoReg: 0
[EX: 0x03A0F025]
data1: 16754872, data2: 0 (forwarding), ALU result: 16754872
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0xAFC40020]
Type: I, Inst: or $a0, 32($s8)
opcode: 0x2B, rs: 30 (value), rt: 4, imm: 0x20
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 0, MemWrite: 1, RegWrite: 1, MemtoReg: 0
[IF: 0x8FC30020]
[PC Update] 0x00000050 -> 0x00000054
32217072> cycle: 9760
[WB: 0x00000044]
Write Register: $s8, Value: 16754896
RegWrite: 1, MemtoReg: 0
[MEM: 0x00000048]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0xAFC40020]
data1: 16754872, data2: -1 (forwarding), ALU result: 16754904
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 0, MemWrite: 1, RegWrite: 1, MemtoReg: 0
[ID: 0x8FC30020]
Type: I, Inst: sw $v1, 32($s8)
opcode: 0x23, rs: 30 (value), rt: 3, imm: 0x20
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 1, MemWrite: 0, RegWrite: 1, MemtoReg: 1
[IF: 0x24020001]
[PC Update] 0x00000054 -> 0x00000058
32217072> cycle: 9761
[WB: 0x00000048]
Write Register: $s8, Value: 16754872
RegWrite: 1, MemtoReg: 0
[MEM: 0x0000004C]
Pass
Store, Address: 0x00FFA8D8, Value: -1
Branch: 0, MemRead: 0, MemWrite: 1, RegWrite: 1, MemtoReg: 0
[EX: 0x8FC30020]
data1: 16754872, data2: -1 (forwarding), ALU result: 16754904
RegDst: 0, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 1, MemWrite: 0, RegWrite: 1, MemtoReg: 1
[ID: 0x24020001]
Type: I, Inst: lw $v0, 1($zero)
```

## 3.fibonacci.bin

```
[ID: 0x00000000]
Type: R, Inst: jr $zero $zero $zero
opcode: 0x0, rs: 0 (value), rt: 0 (value), rd: 0, shamt: 0, funct: 0x0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[IF: 0x8FBE0018]
[PC Update] 0x00000024 -> 0x00000028
32217072> cycle: 2055
[WB: 0x00000028]
Write Register: $sp, Value: 16788769
RegWrite: 1, MemtoReg: 0
[MEM: 0x0000002C]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x8FBE0018]
Type: I, Inst: sll $s8, 24($sp)
opcode: 0x23, rs: 29 (value), rt: 30, imm: 0x18
RegDst: 0, ALUOp: 0, ALUSrc: 1, Branch: 0, MemRead: 1, MemWrite: 0, RegWrite: 1, MemtoReg: 1
[IF: 0x27BD0020]
[PC Update] 0x00000028 -> 0x0000002C
32217072> cycle: 2056
[WB: 0x0000002C]
Write Register: $zero, Value: 16788769
RegWrite: 1, MemtoReg: 0
[MEM: 0x00000030]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x8FBE0018]
```

## 4.factorial.bin

```
32217072> cycle: 5053
[WB: 0x01004CB4]
Write Register: $zero, Value: 0
RegWrite: 1, MemtoReg: 0
[MEM: 0x01004CB8]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x00000000]
Type: R, Inst: sll $zero $zero $zero
opcode: 0x0, rs: 0 (value), rt: 0 (value), rd: 0, shamt: 0, funct: 0x0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[IF: 0x00000000]
[PC Update] 0x01004CC4 -> 0x01004CC8
32217072> cycle: 5054
[WB: 0x01004CB8]
Write Register: $zero, Value: 0
RegWrite: 1, MemtoReg: 0
[MEM: 0x01004CB8]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x00000000]
Type: R, Inst: sll $zero $zero $zero
opcode: 0x0, rs: 0 (value), rt: 0 (value), rd: 0, shamt: 0, funct: 0x0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[IF: 0x00000000]
[PC Update] 0x01004CC8 -> 0x01004CCC
32217072> cycle: 5055
[WB: 0x01004CBC]
Write Register: $zero, Value: 0
RegWrite: 1, MemtoReg: 0
[MEM: 0x01004CC0]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x00000000]
```

## 5.power.bin

```
32217072> cycle: 5053
[WB: 0x01004CB4]
Write Register: $zero, Value: 0
RegWrite: 1, MemtoReg: 0
[MEM: 0x01004CB8]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x00000000]
Type: R, Inst: sll $zero $zero $zero
opcode: 0x0, rs: 0 (value), rt: 0 (value), rd: 0, shamt: 0, funct: 0x0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[IF: 0x00000000]
[PC Update] 0x01004CC4 -> 0x01004CC8
32217072> cycle: 5054
[WB: 0x01004CB8]
Write Register: $zero, Value: 0
RegWrite: 1, MemtoReg: 0
[MEM: 0x01004CBC]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x00000000]
Type: R, Inst: sll $zero $zero $zero
opcode: 0x0, rs: 0 (value), rt: 0 (value), rd: 0, shamt: 0, funct: 0x0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[IF: 0x00000000]
[PC Update] 0x01004CC8 -> 0x01004CCC
32217072> cycle: 5055
[WB: 0x01004CBC]
Write Register: $zero, Value: 0
RegWrite: 1, MemtoReg: 0
[MEM: 0x01004CC0]
Pass
Pass
Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[EX: 0x00000000]
data1: 0, data2: 0 (forwarding), ALU result: 0
RegDst: 1, ALUOp: 2, ALUSrc: 0, Branch: 0, MemRead: 0, MemWrite: 0, RegWrite: 1, MemtoReg: 0
[ID: 0x00000000]
```

### 3. 결론

pipeline 시뮬레이터를 구현하면서 명령어 처리 과정으로 명령어 처리를 여러 단계로 나누어 단계별로 동시에 수행하여 병렬화를 시키는 것을 코드로 구현하는 것이 가장 어려웠다.

구체적으로, IF 단계에서 명령어를 가져오고, ID 단계에서 명령어를 디코드하여 레지스터와 제어 신호를 설정하며, EX 단계에서 연산을 수행하고, MEM 단계에서 메모리 접근을 하며, WB 단계에서 결과를 레지스터 파일에 기록하는 과정을 구현했다. 이를 통해 파이프라인의 주요 기능을 모두 다루었고, forwarding 및 제어 해저드 해결 방법도 적용했다.

그러나 아쉬운 점으로는 무한루프 문제를 완전히 해결하지 못했다는 점이다. 본 시뮬레이터는 프로그램 카운터(PC)가 0xFFFFFFFF 로 설정되었을 때 루프를 종료하도록 설계되었지만, 특정 상황에서 프로그램이 무한루프에 빠질 가능성이 있다.

결론적으로, 본 프로젝트를 통해 파이프라인 프로세서의 기본 원리와 구현 방식을 깊이 있게 이해할 수 있었지만, 무한루프 문제를 완전히 해결하지 못한 것은 아쉬움으로 남는다. 이러한 문제를 해결하기 위한 보다 정교한 해저드 처리 및 분기 예측 기법을 구현하여 시뮬레이터의 완성도를 높이는 방향으로 나아가야 할 것이다.

